

Error Handling in Swift

Module 3

Swift Error Handling

An error (exception) is an unexpected event that occurs during program execution. For example,

```
var numerator = 10
var denominator = 0
// try to divide a number by 0
var result = numerator / denominator // error code
```

Here, we are trying to divide a number. So this type of error causes the abnormal termination of the program.

Swift Error Handling

Steps For Error Handling in Swift

Create an enum that represents the types of errors.

Create a throwing function using the `throws` keyword.

Call the function using the `try` keyword.

Wrap the code with `try` in the `do {...}` block and add the `catch {...}` block to handle all errors.

Swift Error Handling

1. Create enum of Errors

In Swift, we need to create an enum that represents the type of errors we may encounter while writing the program. The enum we create must conform to the **Error** protocol so that we can throw an error value inside the function. Let's see an example,

```
enum DivisionError: Error {  
    case dividedByZero  
}
```

Here, we have created an enum named **DivisionError** with the value **dividedByZero**.

Since **DivisionError** conforms to the **Error** protocol, we'll now be able to throw the error value of the enum.

Swift Error Handling

2. Create a Throwing Function

In order to throw errors, we need to create a throwing function using the **throws** keyword. Then we use the **throw** statement inside the function to throw a particular error represented by the enum. For example,

Here, we have created a throwing function named **division()** using the **throws** keyword. The function throws the **dividedByZero** value of the **DivisionError** enum if **denominator == 0**

Now, based on the value passed during the function call, the function throws an error if the error condition is met.

```
// create throwing function using throws keyword
func division(numerator: Int, denominator: Int) throws {
    // throw error if divide by 0
    if denominator == 0 {
        throw DivisionError.dividedByZero
    }
    ...
}
```

Swift Error Handling

3. Function Call Using try Keyword

In Swift, we use the **try** keyword while calling the throwing function. It indicates that a function can throw an error. For example,

However, the error handling process is still incomplete. If we run the program now, we'll get an error message: An error was thrown and was not caught

So, in order to catch the thrown error, we use the **do-catch** statement.

```
// call throwing function using try keyword  
try division(numerator: 10, denominator: 0)
```


Swift Error Handling

4. Handling Errors Using do-catch Statement

In Swift, we wrap the **try** code in the **do** block and add the **catch** block to handle all errors. For example,

```
//do { try division(numerator: 10, denominator: 0)
...
}
catch DivisionError.dividedByZero {
    // statement
}
```

Here, we have called the throwing function **division()** from within the **do** block and attached the **catch** block to catch the error in case the function throws one.

The above **catch** block is executed based on the enum value of **DivisionError**.

This is the final step to handle possible errors that may occur in our program.

Handle Swift Errors with Do-Catch

In Swift, errors are represented by values of types that conform to the `Error` protocol.

The `do-catch` statement is used to handle errors in a controlled and predictable way. It allows you to specify a block of code that can throw an error and another block of code to handle the error if it's thrown.

Here is an example of using the `do-catch` statement to handle a simple error:

```
enum SimpleError: Error {
    case somethingWentWrong
}
func doSomething() throws -> String
{
    if true {
        throw SimpleError.somethingWentWrong
    }
    return "Success!"
}
do {
    let result = try doSomething()
    print(result)
} catch {
    print("Error: \(error)")
}
```


Handle Swift Errors with Do-Catch

In this example, the `doSomething()` function is declared as a `throwing` function, which means that it can throw errors.

The `do-catch` statement is used so that if it's thrown and the `catch` block is executed, the `error` parameter of the catch block contains the error that was thrown.

```
enum SimpleError: Error {  
    case somethingWentWrong  
}  
func doSomething() throws -> String  
{  
    if true {  
        throw SimpleError.somethingWentWrong  
        return "Success!"  
    }  
}  
do {  
    let result = try doSomething()  
    print(result)  
} catch {  
    print("Error: \(error)")  
}
```

Handle Swift Errors with Do-Catch

It's also possible to catch specific error types by pattern matching on the **error** parameter in the catch block:

```
do { let result = try doSomething()
    print(result)
} catch
SimpleError.somethingWentWrong{
    print("Something went wrong")
} catch let otherError {
    print("Error: (otherError)")
}
```

In this example, the catch block for **SimpleError.somethingWentWrong** will only be executed if the error thrown is of that specific type. If any other error is thrown, it will be caught by the second catch block and the **otherError** constant will be set to the error.

It's also possible to use multiple catch blocks to handle different error types or to handle errors in different ways depending on their specific type or value.

Handle Swift Errors with Do-Catch

It's important to note that using the `try?` or `try!` versions of the `try` keyword can also be used to handle errors, but the `do-catch` statement provides more control and flexibility in handling errors.

It's best practice to always use the `do-catch` statement when working with throwing functions in Swift.

Propagate Swift Errors Using Throwing Functions

If you call a function that throws an error in Swift, you can either handle the error directly with the **do-catch** statement or you can propagate the error by making functions as throwing.

This allows the caller of the function to handle the error, rather than the function itself.

Here is an example of using a throwing function:

```
enum SimpleError: Error {
    case somethingWentWrong
}
func doSomething() throws -> String {
    if true {
        throw SimpleError.somethingWentWrong
    }
    return "Success!"
}
func performAction() throws -> String {
    return try doSomething()
}
do {
    let result = try performAction()
    print(result)
}
catch {
    print("Error: \(error)")
}
```

Propagate Swift Errors Using Throwing Functions

If In this example, the `performAction()` calls `doSomething()`, which can throw an error. However, rather than handling the error within `performAction()`, the function marks itself with `throws` to indicate that it'll propagate the error without handling it.

It's important to note that if a function is marked as throwing, all of its callers must also be marked as throwing or be called within a `do-catch` statement to handle the error.

It's good practice to document the possible errors that a throwing function can throw, so that the caller knows how to handle them.

```
enum SimpleError: Error {
    case somethingWentWrong
}
func doSomething() throws -> String {
    if true {
        throw SimpleError.somethingWentWrong
    }
    return "Success!"
}
func performAction() throws -> String {
    return try doSomething()
}
do {
    let result = try performAction()
    print(result)
}
catch {
    print("Error: \(error)")
}
```


Convert Swift Errors to Optional Values

When working with errors in Swift, sometimes it may be useful to convert an error to an optional value instead of handling it explicitly with a **do-catch** statement or propagating it using a throwing function. This can be useful when the error represents a failure that can be ignored or handled in some other way.

One way to convert errors to optional values is by using the **try?** operator. This operator attempts to evaluate a given expression and returns an optional containing the result if the expression completes without throwing an error. If an error is thrown, `nil` is returned instead.

Here's an example of using the **try?** operator to convert the result of a throwing function to an optional:

```
import Foundation

func loadData(from url: URL) throws -> Data {
    // load data from url
    return Data()
}

let data: Data? = try loadData(from:
URL(string:
"https://www.kodeco.com")) !)
```

Convert Swift Errors to Optional Values

When working with errors in Swift, sometimes it may be useful to convert an error to an optional value instead of handling it explicitly with a **do-catch** statement or propagating it using a throwing function. This can be useful when the error represents a failure that can be ignored or handled in some other way.

One way to convert errors to optional values is by using the **try?** operator. This operator attempts to evaluate a given expression and returns an optional containing the result if the expression completes without throwing an error. If an error is thrown, `nil` is returned instead.

Here's an example of using the **try?** operator to convert the result of a throwing function to an optional:

```
import Foundation

func loadData(from url: URL) throws -> Data {
    // load data from url
    return Data()
}

let data: Data? = try loadData(from:
URL(string:
"https://www.kodeco.com") !)
```

In this example, the **loadData(from:)** function can throw an error if it's unable to load data from the specified URL.

The **try?** operator is used to convert the result of the function to an optional, so that the calling code can handle the case where data isn't successfully loaded.

Convert Swift Errors to Optional Values

When Another way to convert errors to optional values is by using the **try!** operator. This operator attempts to evaluate a given expression and returns the result if the expression completes without throwing an error. If an error is thrown, the program will be terminated with a runtime error.

Here's an example of using the **try!** operator to convert the result of a throwing function to a non-optional:

```
let data = try! loadData(from:
URL(string:
"https://www.kodeco.com") !)
```

In this example, the **loadData(from:)** function can throw an error if it's unable to load data from the specified URL. The **try!** operator is used to convert the result of the function to a non-optional, so that the calling code can handle the case where data isn't successfully loaded.

It's important to note that converting errors to optional values using **try?** or **try!** can make it more difficult to diagnose and fix bugs that occur in your code. If the function can throw an error but the calling code doesn't handle it properly, the error may go unnoticed, leading to unexpected behavior or crashes. It's usually better to use **do-catch** statement or a throwing function to handle errors explicitly.

Use a Result Type for Handling Errors in Swift

The `Result` type is a generic enumeration that can indicate either a success or a failure. It's defined in the Swift standard library and it's a powerful tool for handling errors in a functional way.

The `Result` type has two cases, `.success` and `.failure`. It can be used to represent the outcome of a function that can throw an error.

```
enum Result<Success, Failure: Error>{  
    case success(Success)  
    case failure(Failure)  
}
```

Use a Result Type for Handling Errors in Swift

Here's an example of how you can use the `Result` type to handle errors in a function that reads a file from disk and returns its content as a string:

```
import Foundation
func readFile(at path: String) -> Result<String, Error> {
    do {
        let content = try String(contentsOfFile: path)
        return .success(content)
    } catch {
        return .failure(error)
    }
}
```

Use a Result Type for Handling Errors in Swift

Then you can call the function and handle the result in a switch statement, like this:

```
let result = readFile(at: "/path/to/file.txt")
switch result {
case .success(let content):
    print("Content: \(content)")
case .failure(let error):
    print("Error: \(error)")
}
```

When using the `Result` type, remember that the success and failure cases are associated with specific values and they must be specified at the time of use. Also, it's important to note that the failure case of the `Result` type must conform to the `Error` protocol.